

# SISTEMA DE TOMA DE CONTADORES MULTIMARCA CLOUD

Guía Técnica de Implementación  
**HP • Lexmark • Samsung**

**Arquitectura inspirada en HP Smart Device Services (SDS)**

Agente portable Windows • Backend Node.js • Portal Web • SNMP multimarca

Version 1.0 • 5 de mayo de 2026

## 1. Resumen Ejecutivo y Contexto

Este documento describe la arquitectura, los pasos de implementación y las decisiones técnicas para construir un sistema propio de toma de contadores de impresoras, similar al HP Smart Device Services (SDS), pero diseñado para operar en entornos multimarca (HP, Lexmark y Samsung) con soporte tanto para redes de sede única como para clientes multi-sede.

**El sistema se compone de tres grandes piezas:**

- Agente portable (.exe) que se instala como servicio Windows en el cliente
- Backend cloud en Node.js que recibe, almacena y procesa los datos
- Portal web para configuración, monitoreo y reportes

### Por que construir un sistema propio si ya usamos HP SDS?

HP SDS solo cubre dispositivos HP con firmware FutureSmart. Lexmark y Samsung quedan fuera. Como partners HP, acceden a SDS, pero no pueden extenderlo a otras marcas ni integrar su lógica de negocio propia.

Un sistema propio permite unificar HP + Lexmark + Samsung en una sola plataforma, con facturación automática,

alertas personalizadas, y total control sobre los datos de la flota de clientes.

La inversión se amortiza rápido: elimina el costo de licencias por dispositivo de terceros y diferencia el servicio.

## 2. Lección de HP SDS: Decisiones de Arquitectura

Antes de escribir una línea de código, es clave entender por que HP SDS funciona como funciona. Estas decisiones no son arbitrarias.

### 2.1 El agente inicia todas las conexiones (nunca el cloud)

Este es el patrón de seguridad más importante del sistema. El JAMC (agente HP) no recibe conexiones entrantes del cloud: el mismo hace polling al cloud cada N segundos para preguntar si hay comandos pendientes. Esto significa:

- El cliente NO necesita abrir puertos en su firewall ni hacer port-forwarding
- Todo el tráfico es HTTPS saliente (puerto 443), indistinguible del tráfico web normal
- Si el cloud cae, el agente sigue funcionando en modo local y acumula datos
- Se elimina la superficie de ataque: no hay puertos abiertos esperando conexiones

### Implementación en tu sistema:

El agente usa un ciclo: poll cloud cada 30s -> ejecuta comandos pendientes -> envía lecturas acumuladas.

Usa HTTPS/TLS 1.2 o superior. Nunca HTTP. Certificado X.509 en el servidor.

El agente nunca escucha en ningún puerto local (excepto opcionalmente en localhost para UI de diagnóstico).

## 2.2 Separar identidad del agente vs identidad del dispositivo

HP SDS distingue entre el conector (JAMC, instalado en la PC del cliente) y los dispositivos que ese conector administra. Cada impresora tiene su propio ID en el cloud. Esta separación es fundamental:

- Si una impresora cambia de IP, su identidad en el cloud no cambia (se identifica por MAC + serial)
- Un agente puede gestionar N impresoras. Un mismo cliente puede tener M agentes (multi-sede)
- Los contadores históricos de la impresora sobreviven aunque el agente se reinstale

## 2.3 Cola offline obligatoria

Si el agente pierde internet, los contadores NO se pierden. Se acumulan localmente en SQLite y se sincronizan cuando vuelve la conexión. El cloud los recibe con timestamp original y los incorpora correctamente al histórico. Esto es crítico para la facturación.

## 2.4 El OID universal es el punto de partida, no el destino

Existe un OID estándar (RFC 3805 / Printer MIB) que todos los fabricantes implementan. Sin embargo, cada marca expone datos adicionales en su MIB privada. La estrategia correcta es: intentar el OID estándar primero, y si falla o devuelve 0, intentar los OIDs privados de la marca.

### 3. Fases del Proyecto

|   |                                      |                                                                                                                     |
|---|--------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| 1 | <b>Fase 1: Infraestructura Base</b>  | Repositorio, entorno Node.js, base de datos PostgreSQL + TimescaleDB, Redis, estructura de carpetas y CI/CD básico. |
| 2 | <b>Fase 2: Backend API</b>           | Endpoints de autenticación, gestión de agentes, recepción de lecturas, gestión de dispositivos e impresoras.        |
| 3 | <b>Fase 3: Agente Windows</b>        | Servicio Windows en Node.js con pkg/nexe, scanner SNMP multimarca, cola offline SQLite, comunicación con cloud.     |
| 4 | <b>Fase 4: OIDs por Marca</b>        | Mapeo completo de OIDs HP, Lexmark y Samsung. Lógica de fallback y normalización de datos.                          |
| 5 | <b>Fase 5: Portal Web</b>            | Dashboard React/Vue con mapa de flota, alertas, reportes de contadores, gestión de rangos IP y configuración.       |
| 6 | <b>Fase 6: Seguridad y Hardening</b> | Rotación de tokens JWT, cifrado AES-256 de config local, rate limiting, auditoría de acciones, backup de DB.        |
| 7 | <b>Fase 7: Testing y Piloto</b>      | Pruebas con dispositivos reales de cada marca, piloto con 2-3 clientes, monitoreo de errores SNMP y latencia.       |
| 8 | <b>Fase 8: Producción</b>            | Despliegue en VPS/cloud con Docker Compose, dominio, SSL automático (Let's Encrypt), monitoreo con UptimeRobot.     |

## 4. Fase 1: Infraestructura Base

### 4.1 Stack tecnologico definitivo

| Componente           | Tecnologia                 | Justificacion                                              |
|----------------------|----------------------------|------------------------------------------------------------|
| Runtime backend      | Node.js 20 LTS             | Mismo lenguaje en agente y servidor. Gran ecosistema SNMP. |
| Framework API        | Fastify 4.x                | 50% mas rapido que Express. Validacion de esquemas nativa. |
| Jobs y colas         | BullMQ + Redis             | Reintentos automaticos, visibilidad, scheduling cron.      |
| DB principal         | PostgreSQL 16              | Clientes, agentes, dispositivos, configuracion.            |
| DB series temporales | TimescaleDB                | Extension de Postgres. Compresion automatica de lecturas.  |
| Cache / pub-sub      | Redis 7                    | Sessions, cola offline, rate limiting, pub-sub comandos.   |
| Agente SNMP          | net-snmp (npm)             | Implementacion SNMP v1/v2c/v3 en Node.js puro.             |
| Servicio Windows     | node-windows (npm)         | Instala el proceso Node.js como servicio SCM de Windows.   |
| Empaquetado agente   | pkg o nexe                 | Genera un .exe standalone sin Node.js instalado.           |
| Config cifrada       | node-config + crypto       | AES-256-GCM para credenciales SNMP locales.                |
| Frontend             | React + Vite + TailwindCSS | SPA rapida, componentes reutilizables.                     |
| Contenedores         | Docker + Docker Compose    | Despliegue reproducible en cualquier VPS.                  |

### 4.2 Estructura de carpetas del repositorio

```

/sistema-contadores
  /cloud
    /src
      /api          <- Rutas Fastify (REST endpoints)
      /services     <- Logica de negocio
      /jobs         <- Workers BullMQ (scheduler, sync)
      /db           <- Migrations Knex + repositories
      /ws           <- WebSocket hub (comandos en tiempo real)
    /portal         <- Frontend React (Vite)
    /docker         <- Dockerfiles y docker-compose.yml

```

```
/agent
  /src
    /core      <- Bucle principal del servicio
    /snmp      <- Scanner + OID maps por fabricante
    /sync      <- Cola offline SQLite + uploader
    /config    <- Configuración cifrada local
    /install   <- Scripts instalación Windows
    /build     <- Scripts pkg para generar .exe

  /shared      <- Tipos TypeScript compartidos
  /docs        <- Documentación técnica
```

## 4.3 Base de datos: esquema inicial

Las tablas principales del sistema. Usar Knex.js para migrations versionadas.

```
-- Clientes (empresas que tienen impresoras)
CREATE TABLE clients (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name        VARCHAR(255) NOT NULL,
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

-- Agentes instalados en cada cliente
CREATE TABLE agents (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  client_id   UUID REFERENCES clients(id),
  activation_key VARCHAR(64) UNIQUE,    -- one-time use
  jwt_secret  VARCHAR(128),             -- para validar tokens del agente
  name        VARCHAR(100),
  ip_ranges   JSONB,                   -- [{start:'10.0.1.1', end:'10.0.1.254'}]
  snmp_community VARCHAR(64),          -- cifrada en el agente
  last_seen   TIMESTAMPTZ,
  status      VARCHAR(20) DEFAULT 'pending',
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

-- Dispositivos descubiertos (impresoras)
CREATE TABLE devices (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  agent_id    UUID REFERENCES agents(id),
  ip          INET,
  mac         MACADDR UNIQUE,          -- identificador real del dispositivo
  serial      VARCHAR(100),
  brand       VARCHAR(50),             -- 'hp', 'lexmark', 'samsung'
  model       VARCHAR(100),
  name        VARCHAR(100),
  active      BOOLEAN DEFAULT true,
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

-- Lecturas de contadores (TimescaleDB hypertable)
```

```
CREATE TABLE readings (  
    time          TIMESTAMPTZ NOT NULL,  
    device_id     UUID REFERENCES devices(id),  
    total_pages   INTEGER,  
    mono_pages    INTEGER,  
    color_pages   INTEGER,  
    toner_black   SMALLINT,                -- porcentaje 0-100  
    toner_cyan    SMALLINT,  
    toner_magenta SMALLINT,  
    toner_yellow  SMALLINT,  
    status        VARCHAR(50),  
    offline       BOOLEAN DEFAULT false    -- true = dato de cola offline  
);  
SELECT create_hypertable('readings', 'time');  
-- Compresion automatica despues de 7 dias:  
ALTER TABLE readings SET (timescaledb.compress);  
SELECT add_compression_policy('readings', INTERVAL '7 days');
```

## 5. Fase 2: Backend API (Node.js + Fastify)

### 5.1 Flujo de activación del agente

Este flujo es el más crítico del sistema. Determina cómo un agente nuevo se vincula de forma segura al cloud.

1. El operador accede al portal web y crea un nuevo agente para el cliente X.
2. El backend genera una `activation_key`: string aleatorio de 64 chars con TTL de 24 horas. La guarda en BD con estado 'pending'.
3. El operador instala el .exe en el cliente y ejecuta: `agente.exe --activate <KEY> --server https://tu-dominio.com`
4. El agente hace `POST /api/v1/agents/activate` con la key. El backend valida que exista y no haya expirado.
5. El backend responde con un JWT de larga duración (30 días) y un `refresh_token` (90 días). Marca la key como 'used'.
6. El agente guarda el JWT cifrado en disco (AES-256-GCM con clave derivada del hardware ID de la máquina).
7. Desde ese momento, el agente usa el JWT en el header `Authorization: Bearer <token>` en cada request al cloud.
8. Si el JWT expira, el agente usa el `refresh_token` para obtener uno nuevo automáticamente.

#### Seguridad adicional recomendada:

Revocar el JWT instantáneamente desde el portal (blacklist en Redis con TTL = duración del token).

Limitar la activación a una IP específica si se conoce de antemano (field `allowed_ip` en BD).

Registrar en audit log cada activación: timestamp, IP del agente, user del portal que generó la key.

El hardware ID del agente (derivado de MAC + hostname) actúa como segundo factor implícito.

### 5.2 Endpoints principales del API

| Metodo | Endpoint                                  | Descripción                                       | Auth         |
|--------|-------------------------------------------|---------------------------------------------------|--------------|
| POST   | <code>/api/v1/agents/activate</code>      | Activación con one-time key. Devuelve JWT.        | Key          |
| POST   | <code>/api/v1/agents/refresh</code>       | Renueva JWT con <code>refresh_token</code> .      | RefreshToken |
| GET    | <code>/api/v1/agents/:id/commands</code>  | El agente consulta comandos pendientes (polling). | JWT Agente   |
| POST   | <code>/api/v1/agents/:id/heartbeat</code> | Heartbeat cada 60s con estado del agente.         | JWT Agente   |
| POST   | <code>/api/v1/devices/sync</code>         | El agente sube lecturas (batch)                   | JWT Agente   |



|      |                              |                                                       |            |
|------|------------------------------|-------------------------------------------------------|------------|
|      |                              | de hasta 500 registros).                              |            |
| POST | /api/v1/devices/register     | El agente registra nuevas impresoras descubiertas.    | JWT Agente |
| GET  | /api/v1/clients              | Lista clientes del portal web.                        | JWT Portal |
| GET  | /api/v1/clients/:id/devices  | Lista dispositivos con ultima lectura.                | JWT Portal |
| GET  | /api/v1/devices/:id/readings | Serie temporal de contadores con filtros.             | JWT Portal |
| POST | /api/v1/agents/:id/command   | Portal envia comando al agente (scan, restart).       | JWT Portal |
| PUT  | /api/v1/agents/:id/config    | Actualiza configuracion del agente (rangos IP, etc.). | JWT Portal |

### 5.3 Sistema de comandos bidireccional

El portal necesita poder enviar comandos al agente (forzar scan, cambiar rango IP, reiniciar). Como el agente usa polling saliente, el mecanismo es una cola de comandos en Redis:

```
// Portal envia comando:
POST /api/v1/agents/:id/command
Body: { type: 'FORCE_SCAN', payload: {} }

// Backend guarda en Redis con TTL de 10 minutos:
await redis.lpush(`commands:${agentId}`, JSON.stringify(cmd));
await redis.expire(`commands:${agentId}`, 600);

// El agente en su ciclo de polling pregunta:
GET /api/v1/agents/:id/commands
// Backend devuelve y vacia la cola:
const cmds = await redis.lrange(`commands:${agentId}`, 0, -1);
await redis.del(`commands:${agentId}`);
```

## 6. Fase 3: Agente Windows

### 6.1 Ciclo de vida del agente

El agente opera en un bucle continuo con tres responsabilidades separadas que corren en paralelo:

#### Loop 1: Heartbeat (cada 60 segundos)

Envía al cloud: versión del agente, cantidad de impresoras vistas, errores SNMP recientes, uso de memoria.

Recibe: posibles comandos pendientes, nueva configuración (rangos IP, credenciales SNMP).

Si falla la conexión: no bloquea el agente, solo registra el error y reintenta en el próximo ciclo.

#### Loop 2: Scanner SNMP (cada N minutos, configurable)

Recorre el rango IP configurado haciendo SNMP GET a cada IP.

Para nuevas IPs: identifica marca por sysObjectID, registra en cloud como nuevo dispositivo.

Para IPs conocidas: lee contadores y consumibles, guarda localmente en SQLite.

Estrategia: concurrencia limitada (max 20 requests SNMP simultáneos) para no saturar la red del cliente.

#### Loop 3: Sincronizador (cada 5 minutos)

Lee los registros pendientes de SQLite y los envía al cloud en batches de hasta 500 registros.

Si el cloud responde OK: marca los registros como sincronizados y los elimina de SQLite.

Si el cloud falla: deja los registros en SQLite para el próximo intento (max 7 días de retención local).

Incluye backpressure: si hay más de 10.000 registros pendientes, reduce la frecuencia de scan.

### 6.2 Instalación como servicio Windows

```
// install.js - script de instalación
const { Service } = require('node-windows');

const svc = new Service({
  name:      'ContadorImpresoras',
  description: 'Servicio de monitoreo de impresoras',
  script:    'C:\\ProgramData\\ContadorImpresoras\\agent.js',
  nodeOptions: ['--harmony'],
  env: [{
    name: 'AGENT_DATA_DIR',
    value: 'C:\\ProgramData\\ContadorImpresoras'
  }]
});

svc.on('install', () => svc.start());
svc.install();
```

Para generar el instalador .exe standalone (sin Node.js en el cliente):

```
npx pkg agent/src/main.js --target node20-win-x64 --output  
dist/ContadorImpresoras.exe
```

#### Estructura de archivos del agente en el cliente:

C:\ProgramData\ContadorImpresoras\

|               |                                                           |
|---------------|-----------------------------------------------------------|
| agent.exe     | <- binario compilado                                      |
| config.enc    | <- configuracion cifrada (JWT, rangos IP, community SNMP) |
| local.db      | <- SQLite con cola offline de lecturas                    |
| agent.log     | <- log rotativo (max 10MB, 3 archivos)                    |
| uninstall.bat | <- script de desinstalacion limpia                        |

### 6.3 Identificación de marca por sysObjectID

Cuando el agente encuentra una nueva IP que responde a SNMP, lo primero que hace es leer el OID 1.3.6.1.2.1.1.2.0 (sysObjectID) para identificar el fabricante:

| Prefijo OID        | Fabricante            | Enterprise ID |
|--------------------|-----------------------|---------------|
| 1.3.6.1.4.1.11.*   | HP / Hewlett-Packard  | 11            |
| 1.3.6.1.4.1.641.*  | Lexmark International | 641           |
| 1.3.6.1.4.1.236.*  | Samsung Electronics   | 236           |
| 1.3.6.1.4.1.253.*  | Xerox Corporation     | 253           |
| 1.3.6.1.4.1.1602.* | Canon Inc.            | 1602          |
| 1.3.6.1.4.1.367.*  | Ricoh Company         | 367           |

## 7. Fase 4: OIDs SNMP por Marca

Esta es la fase mas tecnica y la que mas diferencia un sistema generico de uno robusto. La estrategia es: OIDs estandar primero (Printer-MIB RFC 3805), luego OIDs privativos si el estandar falla.

### 7.1 OIDs universales (todos los fabricantes)

Estos OIDs pertenecen al Printer-MIB estandar (RFC 3805) y al Host Resources MIB. Son el punto de partida para cualquier impresora.

| OID                         | Nombre                       | Dato que devuelve                              |
|-----------------------------|------------------------------|------------------------------------------------|
| 1.3.6.1.2.1.1.1.0           | sysDescr                     | Descripción del dispositivo (modelo, firmware) |
| 1.3.6.1.2.1.1.2.0           | sysObjectID                  | Identificador unico del fabricante             |
| 1.3.6.1.2.1.1.5.0           | sysName                      | Nombre del dispositivo                         |
| 1.3.6.1.2.1.43.5.1.1.17.1   | prtGeneralSerialNumber       | Numero de serie                                |
| 1.3.6.1.2.1.43.10.2.1.4.1.1 | prtMarkerLifeCount           | Contador total de paginas (UNIVERSAL)          |
| 1.3.6.1.2.1.43.11.1.1.8.1.x | prtMarkerSuppliesMaxCapacity | Capacidad max del toner x                      |
| 1.3.6.1.2.1.43.11.1.1.9.1.x | prtMarkerSuppliesLevel       | Nivel actual del toner x                       |
| 1.3.6.1.2.1.25.3.5.1.1.1    | hrPrinterStatus              | Estado: idle(3), printing(4), warmup(5)        |
| 1.3.6.1.2.1.25.3.5.1.2.1    | hrPrinterDetectedErrorState  | Bitmask de errores activos                     |

#### Como calcular porcentaje de toner:

prtMarkerSuppliesLevel puede devolver un numero absoluto o porcentaje segun el fabricante.

Si prtMarkerSuppliesMaxCapacity == 100, entonces el Level ya es un porcentaje.

Si MaxCapacity > 100 (ej: 10000 paginas), entonces: % = (Level / MaxCapacity) \* 100.

Algunos fabricantes devuelven -1 o -3 para indicar 'desconocido'. Mapear a null en la BD.

El indice x varia: .1.1 = negro, .1.2 = cyan, .1.3 = magenta, .1.4 = amarillo (orden tipico).

### 7.2 OIDs privativos HP

| OID                                 | Descripción                                              |
|-------------------------------------|----------------------------------------------------------|
| 1.3.6.1.4.1.11.2.3.9.4.2.1.4.1.2.5  | Total de impresiones (contador absoluto - muy confiable) |
| 1.3.6.1.4.1.11.2.3.9.4.2.1.4.1.2.6  | Total paginas color                                      |
| 1.3.6.1.4.1.11.2.3.9.4.2.1.4.1.2.7  | Total paginas monocromaticas                             |
| 1.3.6.1.4.1.11.2.3.9.4.2.1.4.1.2.20 | Numero de serie (alternativo HP)                         |
| 1.3.6.1.4.1.11.2.3.9.1.1.7.0        | Status del motor de impresion                            |

### 7.3 OIDs privados Lexmark

Lexmark implementa Printer-MIB v1 basado en RFC1759 y el estandar 3805. Sus OIDs privados estan bajo el enterprise ID 641.

| OID                          | Descripcion                                                 |
|------------------------------|-------------------------------------------------------------|
| 1.3.6.1.4.1.641.1.1          | Identificador de modelo Lexmark                             |
| 1.3.6.1.4.1.641.2.1.1.1.13.1 | Numero de serie Lexmark (privado)                           |
| 1.3.6.1.4.1.641.2.1.2.1.6.1  | Contador de paginas Lexmark (privado)                       |
| 1.3.6.1.2.1.43.10.2.1.4.1.1  | prtMarkerLifeCount (estandar, tambien soportado)            |
| 1.3.6.1.2.1.43.11.1.1.6.1.*  | prtMarkerSuppliesDescription (descripcion de cada cartucho) |

#### Nota importante sobre Lexmark:

Lexmark provee un MIB pack descargable desde su sitio de soporte ([support.lexmark.com](http://support.lexmark.com), articulo FA615).

Contiene: Lexmark1.MIB, Lexmark2.MIB y archivos de referencia para modelos 2011 en adelante.

Para modelos anteriores a 2011, algunos OIDs privados pueden diferir. Siempre probar en laboratorio.

Lexmark responde correctamente a SNMP v1, v2c y v3. Recomendado usar v2c con community 'public' por defecto.

### 7.4 OIDs Samsung

Samsung fue adquirida por HP en 2017. Los modelos mas nuevos usan firmware HP y pueden responder a OIDs HP. Los modelos Samsung legacy usan el enterprise ID 236.

| OID                            | Descripcion                              |
|--------------------------------|------------------------------------------|
| 1.3.6.1.2.1.43.11.1.1.9.1.1    | Nivel de toner negro (paginas restantes) |
| 1.3.6.1.2.1.43.11.1.1.8.1.1    | Capacidad maxima del cartucho negro      |
| 1.3.6.1.2.1.43.11.1.1.9.1.2    | Nivel toner cyan                         |
| 1.3.6.1.2.1.43.11.1.1.9.1.3    | Nivel toner magenta                      |
| 1.3.6.1.2.1.43.11.1.1.9.1.4    | Nivel toner amarillo                     |
| 1.3.6.1.4.1.236.11.5.11.81.1.0 | Contador total Samsung (privado)         |
| 1.3.6.1.4.1.236.11.5.11.81.2.0 | Contador color Samsung (privado)         |

#### Estrategia de deteccion Samsung legacy vs Samsung-HP:

Leer sysObjectID: si empieza con 1.3.6.1.4.1.11 -> modelo nuevo, usar OIDs HP.

Si empieza con 1.3.6.1.4.1.236 -> modelo Samsung legacy, usar OIDs Samsung.

Para modelos Samsung adquiridos por HP: intentar OIDs HP primero, Samsung como fallback.

Verificar la descripcion del sistema (sysDescr) que puede incluir 'Samsung' o 'HP Samsung'.

## 7.5 Logica de fallback en el agente

El agente implementa un mapa de OIDs por marca y ejecuta una logica de fallback:

```
// Pseudocodigo del scanner multimarca
async function readDevice(ip, community) {
  const brand = await detectBrand(ip, community);
  const oidMap = OID_MAPS[brand] || OID_MAPS['generic'];

  // Contador total: intentar OID privativo, luego estandar
  let totalPages = await snmpGet(ip, oidMap.totalPages);
  if (!totalPages || totalPages === -1) {
    totalPages = await snmpGet(ip, OID_MAPS.generic.totalPages);
  }

  // Toner: leer nivel y capacidad, calcular porcentaje
  const tonerLevel = await snmpGet(ip, oidMap.tonerBlackLevel);
  const tonerMax    = await snmpGet(ip, oidMap.tonerBlackMax);
  const tonerPct    = tonerMax > 100
    ? Math.round((tonerLevel / tonerMax) * 100)
    : tonerLevel; // ya es porcentaje

  return { brand, totalPages, tonerBlack: tonerPct, ... };
}
```

## 8. Fase 5: Portal Web

### 8.1 Vistas principales

| Vista                  | Descripción                                                                                  |
|------------------------|----------------------------------------------------------------------------------------------|
| Dashboard general      | Resumen de la flota: total dispositivos, alertas activas, agentes offline, lecturas del día. |
| Lista de clientes      | Tabla de clientes con total de dispositivos, último contador recibido, estado general.       |
| Detalle de cliente     | Mapa de dispositivos del cliente con estado en tiempo real (online/offline/alerta).          |
| Detalle de dispositivo | Grafico de contadores históricos, niveles de toner, historial de alertas, info del equipo.   |
| Gestión de agentes     | Alta de agentes, generación de activation keys, configuración de rangos IP.                  |
| Reportes               | Exportación de contadores por periodo, por cliente, por modelo. CSV y PDF.                   |
| Configuración          | Usuarios del portal, notificaciones por email, umbrales de alerta de toner.                  |

### 8.2 Sistema de alertas

El backend evalúa cada lectura recibida y genera alertas automáticas según umbrales configurables:

- Toner bajo: cuando el nivel cae por debajo del umbral configurado (default 15%)
- Dispositivo offline: si no se recibe lectura en más de N horas (configurable por cliente)
- Agente offline: si el heartbeat no se recibe en más de 5 minutos
- Error de hardware: cuando hrPrinterDetectedErrorState reporta atasco, sin papel, puerta abierta
- Contador sospechoso: si el total de páginas baja respecto a la lectura anterior (posible reset)

#### Canal de notificaciones:

Email via Nodemailer + SMTP (ej: SendGrid, Amazon SES) para alertas críticas.

Webhook configurable por cliente para integración con sistemas ERP o de ticketing existentes.

En el futuro: notificaciones push via PWA o integración con WhatsApp Business API.

## 9. Fase 6: Seguridad

### 9.1 Seguridad del agente

- Config cifrada: AES-256-GCM. La clave se deriva del hardware ID (MAC + UUID de disco) usando PBKDF2.
- JWT con expiracion de 30 dias y refresh token de 90 dias. Rotacion automatica.
- Blacklist de tokens en Redis: al revocar un agente desde el portal, el JWT queda invalido instantaneamente.
- El agente no escucha puertos. No acepta conexiones entrantes. Solo sale por HTTPS.
- Logs sin datos sensibles: nunca loguear el JWT, community SNMP ni credenciales.

### 9.2 Seguridad del backend

- Rate limiting por IP y por agente: Fastify-rate-limit con Redis como store.
- Validacion estricta de esquemas en cada endpoint con JSON Schema (nativo en Fastify).
- Separacion de roles: JWT de agente != JWT de portal. Scopes distintos, validacion distinta.
- HTTPS obligatorio. HTTP redirige a HTTPS. HSTS habilitado.
- Variables de entorno para todos los secretos. Nunca hardcodeados. Usar .env con dotenv.
- Audit log: cada accion del portal (crear agente, revocar, cambiar config) queda registrada con usuario y timestamp.

### 9.3 Privacidad de datos SNMP

Un punto critico que HP SDS tambien considera: que datos se recolectan y cuales no.

#### Datos que SI se recolectan:

Contadores de paginas (totales, color, monocromatico).

Niveles de toner y consumibles.

Estado del dispositivo (online, offline, error).

Informacion del dispositivo: marca, modelo, serial, IP, MAC.

#### Datos que NUNCA se recolectan:

Contenido de documentos impresos (SNMP no accede al contenido de impresion).

Credenciales de red del cliente.

Informacion personal de usuarios.

Historial de trabajos de impresion (nombres de archivos, usuarios que imprimieron).



## 10. Fase 7: Testing con Dispositivos Reales

### 10.1 Entorno de laboratorio recomendado

Antes del piloto con clientes, armar un entorno de prueba con al menos un dispositivo de cada marca:

- Una impresora HP con firmware FutureSmart (para comparar lectura SNMP vs HP SDS)
- Una impresora Lexmark (preferentemente modelo 2015 o posterior)
- Una impresora Samsung legacy (enterprise ID 236) y una Samsung-HP (post-adquisición)

**Para cada dispositivo, verificar:**

- Que el agente lo descubra correctamente al hacer scan del rango IP
- Que identifique la marca correctamente por sysObjectID
- Que el contador total coincida con el contador visible en el panel físico de la impresora
- Que el nivel de toner coincida con el visible en la interfaz web de la impresora (EWS)
- Que la cola offline funcione: desconectar internet del agente, generar páginas, reconectar y verificar sincronización

### 10.2 Herramientas útiles para debugging SNMP

| Herramienta            | Uso                                                                          |
|------------------------|------------------------------------------------------------------------------|
| snmpwalk (Linux)       | snmpwalk -v2c -c public <IP><br>1.3.6.1.2.1.43 -> vuelca todo el Printer-MIB |
| MIB Browser (OidView)  | GUI para explorar el árbol MIB de un dispositivo real                        |
| Wireshark              | Capturar tráfico SNMP UDP/161 para ver exactamente que devuelve cada OID     |
| iReasoning MIB Browser | Herramienta gratuita, soporta v1/v2c/v3, carga MIBs privados                 |
| PRTG Free (trial)      | Monitoreo rápido para validar que los OIDs devuelven datos correctos         |

## 11. Fase 8: Despliegue en Producción

### 11.1 docker-compose.yml mínimo

```
version: '3.9'
services:
  api:
    build: ./cloud
    ports: ['3000:3000']
    env_file: .env
    depends_on: [postgres, redis]
```

```
portal:
  build: ./cloud/portal
  ports: ['80:80', '443:443']

postgres:
  image: timescale/timescaledb:latest-pg16
  volumes: ['pgdata:/var/lib/postgresql/data']
  environment:
    POSTGRES_PASSWORD: ${DB_PASSWORD}

redis:
  image: redis:7-alpine
  volumes: ['redisdata:/data']
  command: redis-server --appendonly yes

nginx:
  image: nginx:alpine
  ports: ['80:80', '443:443']
  volumes:
    - ./nginx.conf:/etc/nginx/nginx.conf
    - certbot-certs:/etc/letsencrypt

volumes:
  pgdata:
  redisdata:
  certbot-certs:
```

## 11.2 Checklist de lanzamiento

9. Dominio configurado con DNS apuntando al servidor
10. Certificado SSL via Certbot/Let's Encrypt con auto-renovación
11. Variables de entorno en .env: DB\_PASSWORD, JWT\_SECRET, REDIS\_URL, SMTP\_\*
12. Backup automático de PostgreSQL: cron diario con pg\_dump a S3 o similar
13. Monitoreo de uptime: UptimeRobot o Better Uptime con alertas por email/SMS
14. Log management: PM2 con log rotation o envío a Papertrail/Logtail
15. Piloto con 2-3 clientes seleccionados durante 2 semanas antes de expansión
16. Documentación del proceso de instalación del agente para el equipo técnico

## 12. Roadmap de Evolucion

Una vez estabilizado el sistema base, estas son las mejoras naturales ordenadas por impacto:

| Prioridad | Mejora                       | Impacto                                                                                                   |
|-----------|------------------------------|-----------------------------------------------------------------------------------------------------------|
| Alta      | Integracion con HP SDS API   | Para impresoras HP FutureSmart 4.9+, usar SDS como fuente complementaria. Doble validacion de contadores. |
| Alta      | Exportacion automatica a ERP | Webhook o CSV programado para sistemas de facturacion por paginas.                                        |
| Media     | App movil (React Native)     | Alertas push, visualizacion de flota desde celular para tecnicos.                                         |
| Media     | Soporte SNMPv3               | Autenticacion y cifrado SNMP para clientes con politicas de seguridad estrictas.                          |
| Media     | Soporte Xerox y Canon        | Agregar OID maps para estos fabricantes. Enterprise IDs: Xerox=253, Canon=1602.                           |
| Baja      | Machine Learning             | Prediccion de reemplazo de toner basada en historial de consumo del dispositivo.                          |
| Baja      | White-label                  | Personalizar el portal con logo y colores del cliente para reventa del servicio.                          |

**Este documento es confidencial y de uso interno.**

No distribuir a terceros sin autorizacion.